

PID-Based Temperature Control System ☒

A closed-loop temperature-control system using an Arduino UNO, LM35 temperature sensor, IRF520 MOSFET driver, and a 12 V DC bulb. The system regulates temperature around a 32 °C setpoint by varying heater power through PWM.

[Platform](#)

[Language](#)

[Analysis](#)

[License](#)

Overview

This project demonstrates closed-loop temperature control using PID control. The Arduino measures temperature through an LM35 sensor, calculates the error from the desired setpoint, and controls a 12 V DC bulb through PWM and an IRF520 MOSFET module.

The control loop is:

Measure → Compare → Calculate → Actuate → Measure again

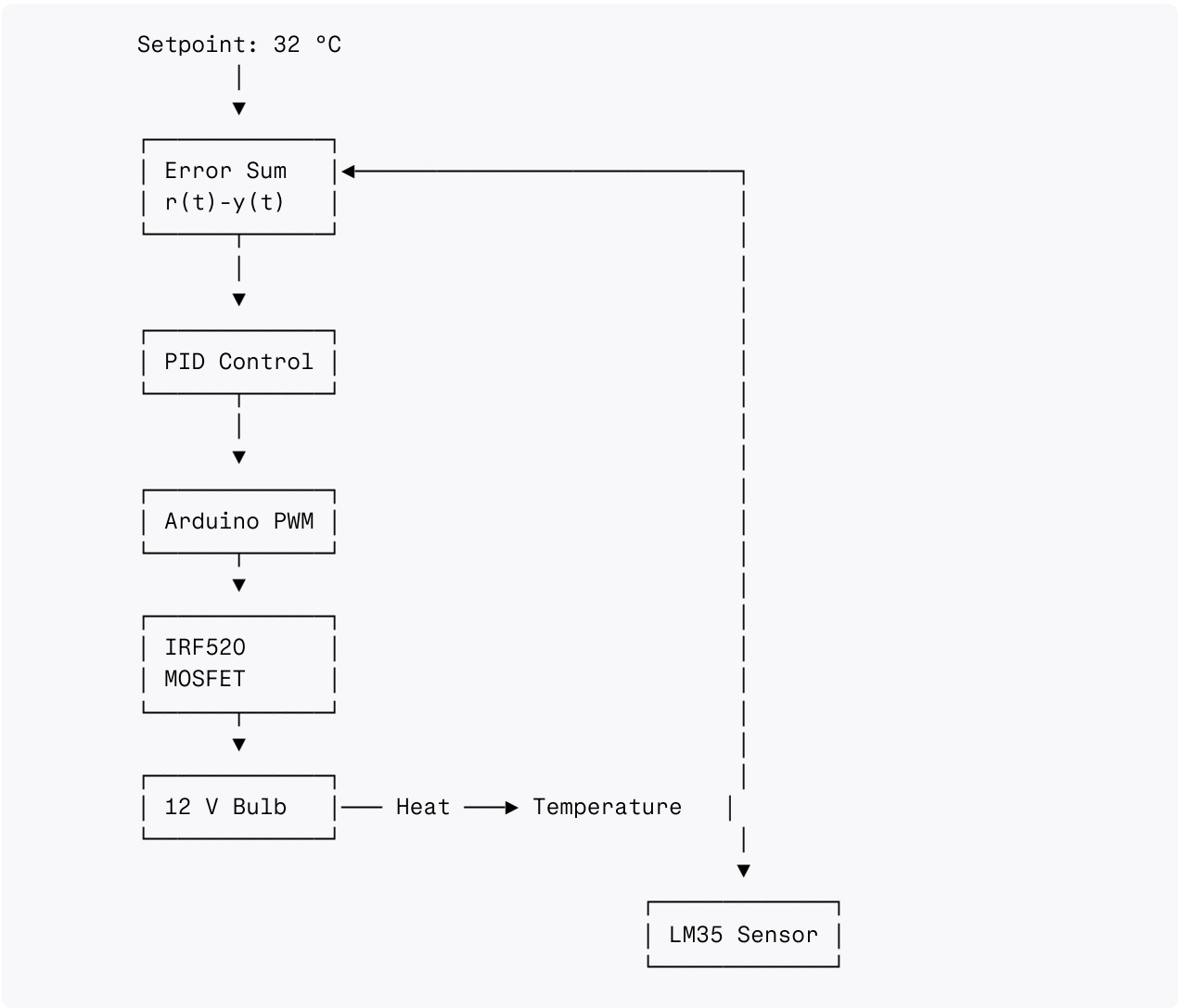
Features

- Arduino UNO-based temperature control.
- LM35 analog temperature sensing.
- PID controller implementation.
- PWM heater-power control.
- IRF520 MOSFET switching.
- MATLAB transfer-function analysis.
- Open-loop, closed-loop, PI, PD, and PID comparison.
- Integral anti-windup.
- Serial Monitor output for temperature and PWM values.

Project Objective

The objective is to maintain the system temperature near a target setpoint of 32 °C by automatically adjusting the power supplied to a DC bulb.

System Block Diagram



Hardware Requirements

Component	Quantity	Purpose
Arduino UNO	1	Controller and ADC/PWM generation
LM35 temperature sensor	1	Temperature measurement
IRF520 MOSFET module	1	Heater switching
12 V DC bulb	1	Heating actuator
12 V DC power supply	1	Bulb supply
Breadboard or terminal board	1	Prototyping
Jumper wires	As required	Connections
Fuse or current protection	1	Recommended for safety

Pin Connections

Arduino pin	Connected to	Description
A0	LM35 Vout	Analog temperature signal
5 V	LM35 Vs	Sensor supply
GND	LM35 GND, MOSFET GND, supply negative	Common ground
D9	IRF520 signal/gate input	PWM control signal
12 V positive	Bulb positive terminal	External bulb supply
Bulb negative terminal	MOSFET drain/load terminal	Switched heater path
MOSFET source/GND	12 V supply negative	Return path

Verify the exact labels on your IRF520 module before connecting power. Never connect the bulb directly to an Arduino pin.

Electrical Connection Summary

```
12 V (+) — Bulb (+)
Bulb (-) — MOSFET Drain/Load
MOSFET Source/GND — 12 V (-)
Arduino GND — 12 V (-)
Arduino D9 — MOSFET Signal/Gate
LM35 Vout — Arduino A0
LM35 Vs — Arduino 5 V
LM35 GND — Arduino GND
```

Control Theory

The PID controller is defined as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

where $e(t) = r(t) - y(t)$ is the temperature error.

Initial controller parameters used in the firmware:

```
Setpoint = 32.0 °C
Kp        = 15.0
Ki        = 0.30
Kd        = 1.00
```

The integral term is limited to reduce integral windup. The PWM output is constrained between 30 and 255.

Plant Model

The MATLAB analysis uses the simplified first-order plant:

$$G(s) = \frac{1}{10s + 1}$$

The actual thermal response depends on bulb power, enclosure design, ambient temperature, sensor location, and heat loss.

Repository Structure

```
PID-Based-Temperature-Control-System/  
├─ README.md  
├─ LICENSE  
├─ .gitignore  
├─ firmware/  
│   └─ PID_Temperature_Control/  
│       └─ PID_Temperature_Control.ino  
├─ matlab/  
│   └─ MATLAB_PID_Analysis.m  
├─ hardware/  
│   ├── bill-of-materials.md  
│   └─ wiring-table.md  
├─ docs/  
│   ├── control-theory.md  
│   └─ experimental-procedure.md  
├─ results/  
│   └─ README.md  
└─ images/  
    └─ README.md
```

Software Requirements

Arduino

- Arduino IDE.
- Arduino UNO board package.
- USB cable.
- Serial Monitor at 9600 baud.

MATLAB

- MATLAB.
- Control System Toolbox.
- Simulink, if a Simulink model is used.

Arduino Installation

1. Open `firmware/PID_Temperature_Control/PID_Temperature_Control.ino` in Arduino IDE.
2. Select Tools → Board → Arduino UNO.
3. Select the correct serial port.
4. Compile the sketch.
5. Upload it to the Arduino.
6. Open Serial Monitor at 9600 baud.
7. Check the sensor readings before enabling the 12 V supply.

MATLAB Analysis

Run the following script in MATLAB:

```
matlab/MATLAB_PID_Analysis.m
```

The script generates open-loop, closed-loop, PI, PD, and PID step responses, pole-zero maps, and a performance comparison table.

Firmware Operation

The firmware performs the following steps:

1. Reads the LM35 analog voltage.
2. Converts the ADC value to temperature.
3. Calculates the sampling interval.
4. Computes proportional, integral, and derivative terms.
5. Limits integral accumulation.
6. Constrains the control signal.
7. Writes the PWM signal to pin D9.
8. Prints monitoring data through Serial Monitor.

Example Serial Output

```
Temperature: 28.43 C | Setpoint: 32.00 C | Error: 3.57 C | PWM: 58  
Temperature: 29.01 C | Setpoint: 32.00 C | Error: 2.99 C | PWM: 47  
Temperature: 30.12 C | Setpoint: 32.00 C | Error: 1.88 C | PWM: 35
```

These values are illustrative. Actual values depend on the experimental setup.

PID Tuning

1. Start with $K_i = 0$ and $K_d = 0$.
2. Increase K_p until the response is sufficiently fast.
3. Increase K_i gradually to reduce steady-state error.
4. Add a small K_d value to improve damping if required.
5. Record rise time, settling time, overshoot, and steady-state error.
6. Change one parameter at a time.

Experimental Procedure

1. Disconnect the 12 V supply.
2. Check all wiring and sensor polarity.
3. Upload the Arduino firmware.
4. Verify plausible ambient-temperature readings.
5. Open Serial Monitor at 9600 baud.
6. Enable the 12 V supply after completing safety checks.
7. Record temperature and PWM values.
8. Stop the experiment if any component overheats.
9. Save cleaned data in the `results/` directory.
10. Compare experimental data with MATLAB results.

Safety Notes

- The bulb, MOSFET, and wiring can become hot.
- Use a suitable 12 V current-rated power supply.
- Do not power the bulb from the Arduino 5 V pin.
- Maintain a common ground between Arduino and MOSFET control circuitry.
- Check the current and thermal ratings of the MOSFET module.
- Keep flammable materials away from the heater.
- Disconnect power before changing wiring.
- Use a fuse or current-limiting protection.
- Do not leave the system unattended.

Limitations

- The plant model is a simplified first-order approximation.
- The LM35 conversion assumes a nominal 10 mV/°C scale factor.
- ADC reference-voltage variation affects measurement accuracy.

- The derivative term can respond to sensor noise.
- The PWM lower bound may prevent complete heater shutoff.
- The IRF520 may not be ideal at Arduino gate voltage.
- The firmware does not currently include independent hardware over-temperature protection.
- Actual behavior depends on the physical thermal arrangement.

Suggested Improvements

- Add an over-temperature cutoff.
- Add a hardware fuse and emergency disconnect.
- Use a logic-level MOSFET with a suitable current rating.
- Add sensor averaging or filtering.
- Implement non-blocking timing instead of `delay()`.
- Stream comma-separated data for automatic analysis.
- Add an LCD or OLED display.
- Compare simulated and experimental responses.
- Add measured plots and hardware photographs.

Author

Sritam

Electronics/Engineering Student, NIT Rourkela

License

This project is released under the MIT License. See [LICENSE](#).

Disclaimer

This project is intended for educational and experimental use. Verify all electrical and thermal designs before operating the system.